



Faculty of Engineering and Technology
Electrical and Computer Engineering Department
Interfacing Techniques - ENCS4380

Micromouse Phase 1 Simulation Report

Prepared by:

Lara Fuqaha	1220071
Hala Sabobeh	1220322
Nour Omari	1221212
Shahd Loai	1211019

Instructor:

Dr. Wasel Ghanem

Section 2

April 2026

Table of Contents

Table of FiguresII

Introduction..... 3

Algorithm Descriptions..... 3

Performance Results 4

Simulation Heatmaps 6

Logic Flowchart for Optimized Algorithm..... 13

Conclusion 14

Table of Figures

Figure 1: Wall-following heatmap for a 16x16 maze.....	6
Figure 2: Wall-following heatmap for a 8x8 maze.....	7
Figure 3: Standard floodfill heatmap for a 16x16 maze	8
Figure 4: Standard floodfill heatmap for a 8x8 maze	9
Figure 5: Optimized floodfill heatmap for a 16x16 maze.....	10
Figure 6: Optimized floodfill heatmap for a 8x8 maze.....	12
Figure 7: Logic flowchart of the Optimized Floodfill algorithm.....	13

1. Introduction

This report documents the simulation phase of the Micromouse project. Three navigation algorithms were implemented and tested in the mms simulator on both 8×8 and 16×16 mazes. The goal is to compare performance in terms of steps taken, cells visited, and number of turns.

The three algorithms implemented are:

1. Wall-Following (Left/Right-hand rule)
2. Standard Floodfill
3. Optimized Floodfill (Modified Floodfill with Speed Run)

2. Algorithm Descriptions

- **Wall-Following (Left-Hand/Right-Hand Rule)**

The wall-following method represents the most basic navigation strategy. Using the left-hand rule, the robot (or mouse) continuously keeps its left side in contact with a wall and follows it through the maze. Similarly, the right-hand rule requires the right side to maintain contact with a wall at all times. Although these rules are straightforward to implement, they are not without limitations. They may fail in mazes that contain isolated walls or loops, and they do not ensure that the shortest route to the goal is found.

Implementation language: C++

- **Standard Floodfill**

The Floodfill algorithm assigns a distance value to every cell in the maze, where the center cells receive a value of 0 and every other cell receives a value equal to its minimum distance to the center. The mouse always moves to the neighboring cell with the lowest flood value. When a new wall is discovered, all flood values are recalculated using BFS from the center outward. This guarantees the mouse always takes the currently known shortest path.

Implementation language: Python

- **Optimized Floodfill**

The Optimized Floodfill extends standard Floodfill using a three-phase approach. In Phase 1, the mouse navigates to the center using Floodfill while mapping all discovered walls. In Phase 2, it returns to the start, continuing to build the wall map. In Phase 3 (Speed Run), the mouse runs to the center on the fully known shortest path with no detours or recalculations, reaching the center significantly faster than standard Floodfill. In real micromouse competitions, only the speed run time is scored, making this the superior strategy.

Implementation language: Python

3. Performance Results

The table below shows the performance metrics recorded from the Stats tab of the mms simulator for each algorithm across multiple mazes. Lower values indicate better performance.

Algorithm	Maze	Size	Reached Center	Distance to Center	Unique Cells Visited
Wall-following left	Example 1	16x16	No	–	–
	Example 2	16x16	No	–	–
	Example 3	16x16	No	–	–
	Test 1	8x8	Yes	20	12
	Test 2	8x8	Yes	32	13
Wall-following right	Example 1	16x16	No	–	–
	Example 2	16x16	No	–	–
	Example 3	16x16	No	–	–
	Test 1	8x8	Yes	32	17
	Test 2	8x8	Yes	16	8
Standard floodfill	Example 1	16x16	Yes	276	118
	Example 2	16x16	Yes	68	35
	Example 3	16x16	Yes	232	106
	Example 4	16x16	Yes	576	199
	Test 1	8x8	Yes	36	18
	Test 2	8x8	Yes	20	9
Optimized floodfill	Example 1	16x16	Yes	168 – speed run 644 – all phases	203
	Example 2	16x16	Yes	64 – speed run 200 – all phases	61
	Example 3	16x16	Yes	140 – speed run 612 – all phases	167
	Example 4	16x16	Yes	120 – speed run 468 – all phases	143
	Test 1	8x8	Yes	20 – speed run 80 – all phases	17
	Test 2	8x8	Yes	12 – speed run	11

Time to center is measured as the total distance recorded by the mms simulator's Stats tab, which counts the number of moves and turns made by the mouse from the starting cell to the center. This metric is used as a proxy for time since each move and turn takes equal time in simulation. Unique cells visited represents the number of distinct cells the mouse entered during its run, extracted directly from the algorithm's visited counter. Lower values for both metrics indicate better performance. For the Optimized Floodfill, Steps to Center refers to the Phase 3 Speed Run distance only, while Total Distance includes all three phases combined.

Discussion of Results

The results clearly demonstrate the trade-offs between the three navigation approaches. Wall-following algorithms failed to reach the center on all 16×16 mazes due to isolated wall structures, where the start and center are not connected to the same continuous wall. This is a fundamental limitation of the algorithm. On 8×8 mazes, wall-following succeeded but required more steps than floodfill, and the path taken was not optimal.

Standard Floodfill successfully solved all mazes on both sizes. By dynamically recalculating flood values whenever a new wall was discovered, it always followed the best known path. However, it still took detours early in the run before the maze was fully mapped.

The Optimized Floodfill demonstrated the most efficient pathfinding. Although its total distance across all three phases was higher than standard floodfill, its Phase 3 speed run consistently achieved the shortest distance to center across all mazes. For example, on Example 4 (the most complex 16×16 maze), standard floodfill required 576 steps while the optimized speed run reached the center in only 120 steps, a 79% improvement. This confirms that the three-phase approach is significantly superior when the speed run time is the primary metric.

The following heatmaps were captured from the mms simulator showing the exploration patterns of each algorithm.

Figure 1 below shows the heatmap for a 16x16 maze using wall-following (left rule) algorithm.



6

Figure 2 below shows the heatmap for a 8x8 maze using wall-following (left rule) algorithm.

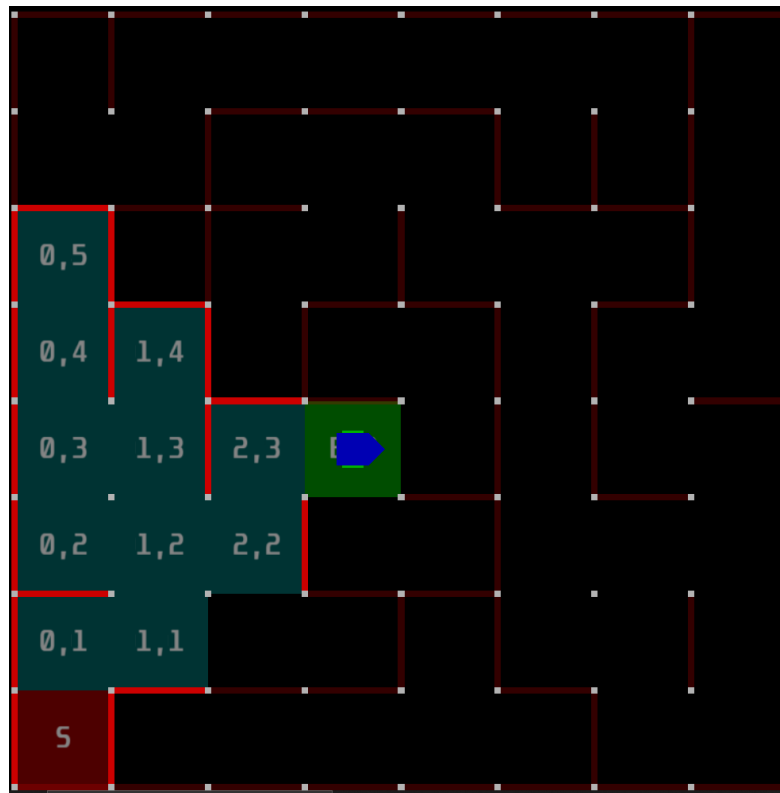


Figure 2: Wall-following heatmap for a 8x8 maze

The heatmap shows the wall-following algorithm successfully reaching the center (green cell) on an 8x8 maze. The colored cells represent the path taken by the mouse, which followed the left wall from the starting cell (bottom-left) to the center. Only 12 unique cells were visited out of 64 total, demonstrating that on simpler mazes with connected wall structures, wall-following can find a path efficiently. However, the path is not optimal as the algorithm has no mechanism to seek the shortest route.

- **Standard Floodfill**

The figure below shows the heatmap for a 16x16 maze using standard floodfill algorithm.

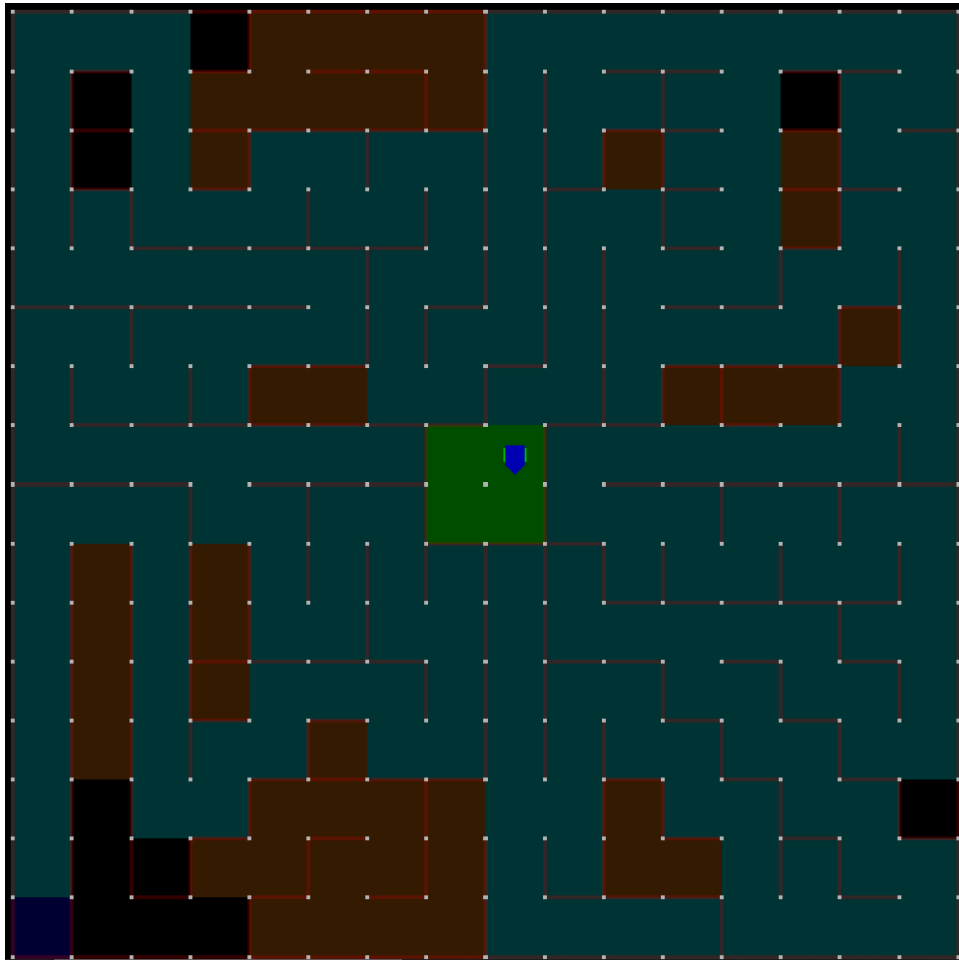


Figure 3: Standard floodfill heatmap for a 16x16 maze

The heatmap shows the Standard Floodfill algorithm solving a 16×16 maze. The blue cells represent visited cells, brown cells are areas explored but backtracked from when better paths were found, and the green cell is the center goal reached. The blue cell at the bottom-left is the starting position. Unlike wall-following, the floodfill successfully navigates the entire maze by dynamically recalculating flood values whenever a new wall is discovered, always redirecting toward the lowest flood value. The scattered exploration pattern across the maze reflects the algorithm rerouting multiple times before finding the optimal path to the center.

The figure below shows the heatmap for a 8x8 maze using standard floodfill algorithm.

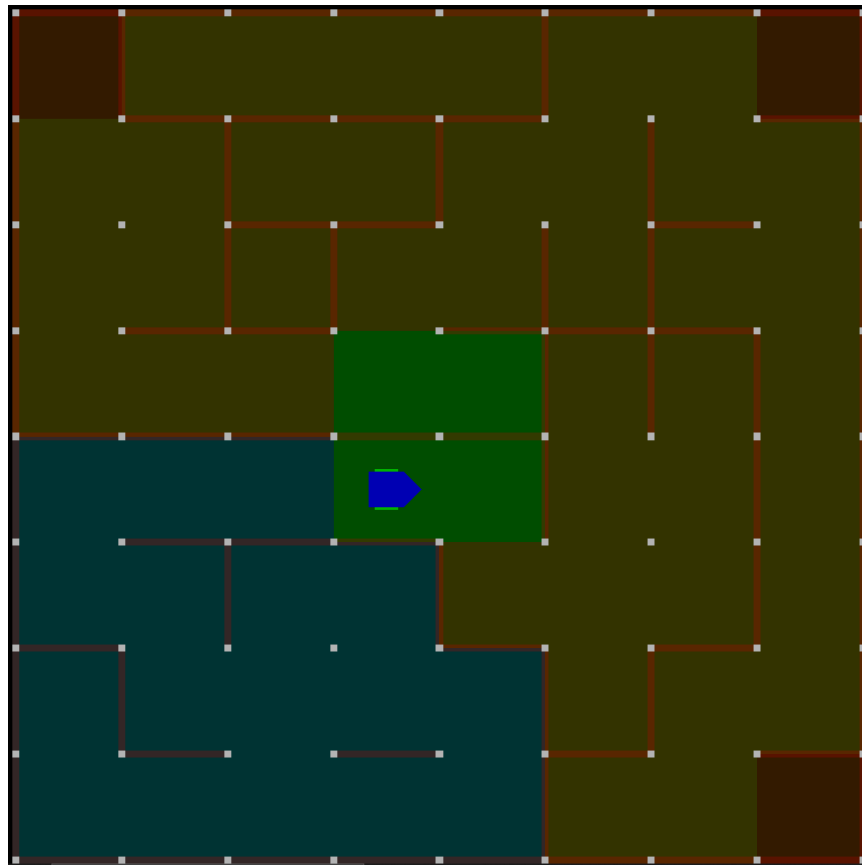


Figure 4: Standard floodfill heatmap for a 8x8 maze

The heatmap shows the Standard Floodfill algorithm solving an 8x8 maze. The teal cells represent the path taken by the mouse, brown cells are explored areas where the algorithm backtracked after discovering walls, and the green cells mark the center goal. The algorithm successfully reached the center by continuously recalculating flood values. Compared to the 16x16 maze, the exploration coverage is more complete relative to maze size, as the smaller maze requires the mouse to traverse a larger proportion of cells to find the optimal path.

- **Optimized Floodfill**

The figure below shows the heatmap for a 16x16 maze using optimized floodfill algorithm.

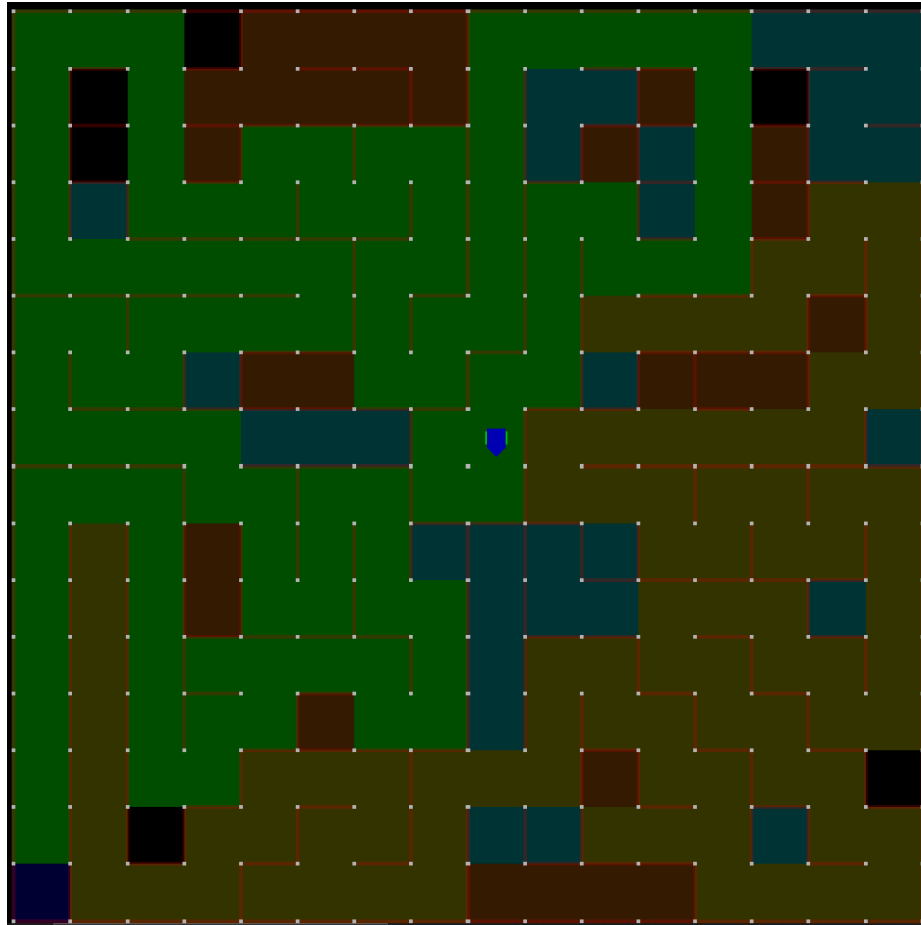


Figure 5: Optimized floodfill heatmap for a 16x16 maze

The heatmap shows the three phases of the Optimized Floodfill algorithm on a 16×16 maze. The blue cells represent Phase 1 (exploration run to center), brown cells represent Phase 2 (return run to start), and green cells represent Phase 3 (speed run to center). The extensive coverage in Phases 1 and 2 reflects the algorithm building a complete wall map, which enables the Phase 3 speed run to follow the direct optimal path with no detours. The near-complete maze coverage confirms that by the time the speed run begins, the mouse has sufficient knowledge to navigate optimally, explaining why the speed run distance (120 steps) is significantly lower than the standard floodfill distance (576 steps) on the same maze.

The previous Example 4 was selected as the worst-case maze, as it produced the highest distance to center across all tests (576 steps for standard floodfill, 199 unique cells visited), indicating a highly complex structure with multiple dead-ends and long corridors.

How the Optimized Floodfill prevents infinite looping:

When the mouse encounters a dead-end, all neighboring cells have flood values of INF (unreachable). Instead of stopping, the algorithm recalculates flood values using BFS from the center through all known open paths. This forces the flood values to update and redirects the mouse back through the last known valid path. Since flood values are always recalculated based on actual wall knowledge, the mouse can never be permanently stuck, it will always find a lower flood value neighbor to move toward.

Additionally, the visited counter tracks how many times each cell has been entered. When multiple neighbors have equal flood values, the mouse prefers the least-visited one, preventing it from oscillating between the same two cells indefinitely.

The figure below shows the heatmap for a 8x8 maze using optimized floodfill algorithm.

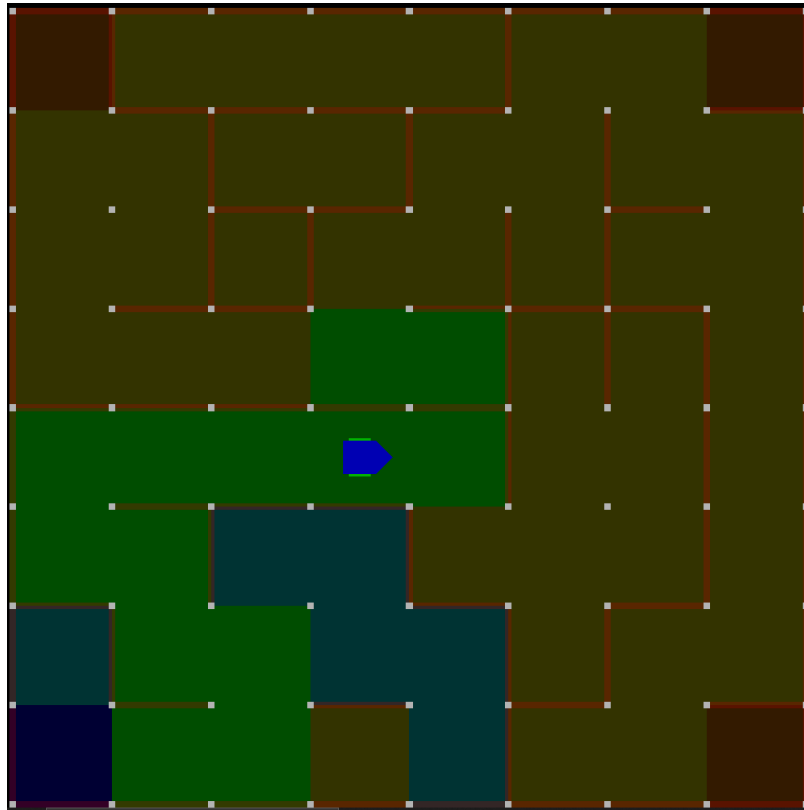


Figure 6: Optimized floodfill heatmap for a 8x8 maze

The heatmap in Figure 6 above shows the Optimized Floodfill algorithm solving an 8x8 maze. The dark blue cell represents the starting position, blue and brown cells show Phase 1 and Phase 2 exploration, and green cells mark the Phase 3 speed run path to the center. The compact exploration pattern compared to the 16x16 maze reflects the smaller maze size, where fewer cells need to be visited before a complete wall map is built. The speed run (green path) takes the most direct route to the center, confirming the algorithm's effectiveness even on smaller mazes.

5. Logic Flowchart for Optimized Algorithm

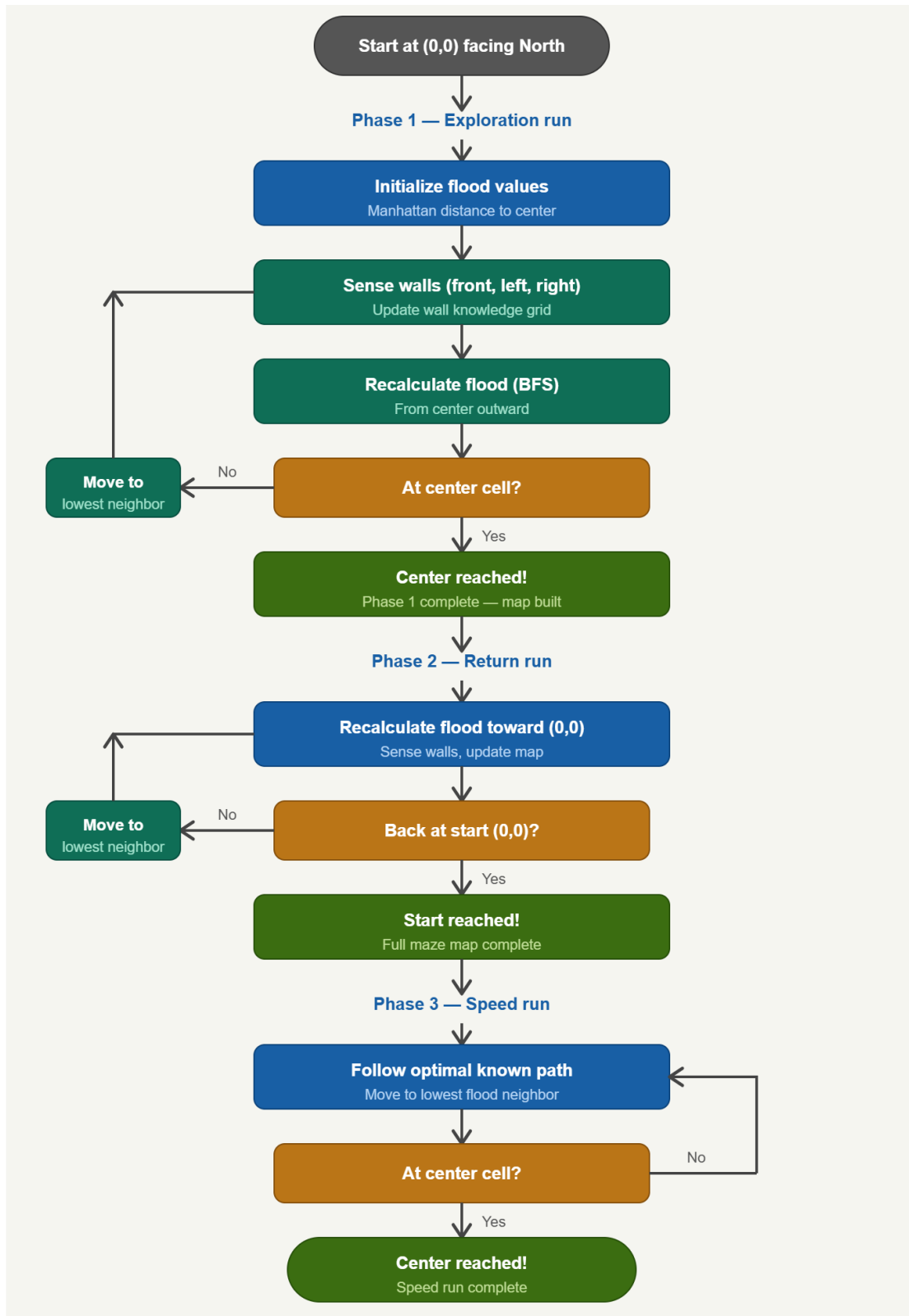


Figure 7: Logic flowchart of the Optimized Floodfill algorithm

6. Conclusion

This simulation phase demonstrated clear performance differences between the three navigation approaches. Wall-Following failed to reach the center on all 16×16 mazes due to its inability to handle isolated wall structures, and while it succeeded on simpler 8×8 mazes, it never found the optimal path. Standard Floodfill solved all mazes by dynamically recalculating distances when new walls were discovered, consistently following the shortest known path. The Optimized Floodfill proved the most efficient, with its Phase 3 speed run outperforming Standard Floodfill by up to 79% on complex mazes (120 vs 576 steps on Example 4). Overall, algorithm complexity directly correlates with performance. The more intelligent the navigation strategy, the fewer steps required to reach the center. These results confirm that the three-phase Optimized Floodfill is the superior strategy for real micromouse competitions where speed run time is the primary metric, and will inform sensor placement and chassis design decisions in Phase 2.